Ciências da Computação – 6º Semestre Noturno (NACCOMP6)

**Entrega 02 – Programação e Controle de Dispositivos no Arduino**

**Sweet Spot**

Arthur Paltrinieri (24026559)
Beatriz de Souza Santos Rio Branco (23025365)
Fabiano Henrique Chou (24025991)
Marcelo Henrique de Souza (23025665)

2026

**RESUMO**

O projeto Sweet Spot propõe uma plataforma de treinamento solo de precisão no tênis, utilizando alvos luminosos em uma maquete e prevendo integração com visão computacional e aplicação web. Esta entrega aborda a programação e o controle de dispositivos com Arduino, com ênfase no acionamento de LEDs por circuitos integrados, no uso de estruturas de decisão e repetição e na leitura de uma entrada digital. As fotografias apresentadas registram a montagem física, um Arduino Uno conectado e LEDs iluminados na superfície de treino. O código-fonte incluído ao final do documento é uma implementação demonstrativa para dois registradores 74HC595 e 16 canais, com botão de teste como entrada digital. A correspondência do código com a montagem fotografada, assim como a leitura efetiva de um sensor, ainda precisa ser validada em bancada.
Palavras-chave: Arduino. LEDs. Registradores de deslocamento. Controle digital. Sweet Spot.

## 1 INTRODUÇÃO

A Sweet Spot combina recursos de Internet das Coisas (IoT), visão computacional e gamificação para criar uma experiência de treino de precisão no tênis. No protótipo, uma parede com LEDs representa regiões-alvo e bolas de isopor substituem as bolas de tênis utilizadas em uma eventual estrutura de lançamento. A proposta global prevê que duas webcams e um modelo YOLO identifiquem a trajetória da bola e subsidiem o cálculo da posição de impacto.

A presente entrega tem escopo específico: documentação do subsistema embarcado que aciona os alvos luminosos e de uma rotina demonstrativa de leitura digital. As funcionalidades de rastreamento, cálculo da precisão geométrica, API Node.js e banco MySQL pertencem à arquitetura mais ampla e não são implementadas pelo sketch apresentado.

## 2 OBJETIVOS

O objetivo geral é apresentar uma solução programável em Arduino para controlar ao menos um atuador e ler ao menos uma entrada digital, de acordo com o enunciado da Entrega 02.

São objetivos específicos: demonstrar a formação de alvos por LEDs; utilizar variáveis, estruturas condicionais e de repetição; aplicar leitura digital com tratamento de ruído de botão; organizar uma rotina de pontuação demonstrativa e documentar a montagem por registros fotográficos.

## 3 COMPONENTES E ARQUITETURA DO PROTÓTIPO

| Elemento | Função na solução | Situação observável |
|---|---|---|
| Arduino Uno | Unidade de controle eletrônico | Placa identificável na fotografia 3 |
| Protoboards e circuitos integrados | Montagem e distribuição de sinais aos pontos luminosos | Duas protoboards com diversos CIs na fotografia 1 |
| Matriz de LEDs | Representação visual dos alvos | LEDs acesos na fotografia 2 |
| Fiação e alimentação | Interligação elétrica | Cabos visíveis; pinagem e tensões não aferidas |
| Entrada digital de teste | Simulação do evento de acerto no código de exemplo | Botão externo não identificado nas fotografias |

A referência inicial enviada pela equipe mostra o uso de registradores 74HC595, enquanto as novas imagens exibem uma montagem com múltiplos circuitos integrados. Não é possível identificar com segurança cada CI nem reconstruir a pinagem exclusivamente pelas fotografias. Por isso, o código de 16

canais deste documento deve ser compreendido como implementação demonstrativa a ser adaptada à quantidade de saídas e à disposição real dos LEDs.

## 4 LÓGICA DE FUNCIONAMENTO

### 4.1 Inicialização

A função setup() configura três saídas digitais para transmissão de dados e um pino de entrada com resistor pull-up interno, inicializa a comunicação serial em 9600 bit/s e apresenta o primeiro alvo.

### 4.2 Controle dos alvos luminosos

Uma variável determina a largura do alvo, expressa como a quantidade de LEDs consecutivos iluminados. A função gerarAlvo() define uma posição inicial pseudoaleatória, monta uma máscara de 16 bits e transmite os dados aos registradores por meio da função shiftOut().

### 4.3 Níveis de dificuldade

O programa de referência aceita os comandos 1 (fácil, seis LEDs), 2 (médio, quatro LEDs) e 3 (difícil, dois LEDs) pelo Monitor Serial. O alvo visual é reduzido nas dificuldades maiores. Essas larguras não comprovam as dimensões físicas dos alvos da maquete.

### 4.4 Leitura da entrada digital

O código demonstra a utilização de um botão conectado ao pino D2 e ao GND, com INPUT_PULLUP. O acionamento é detectado por transição estável para nível baixo (LOW), com debounce de 35 milissegundos. Esse botão é um recurso didático para o requisito de leitura de entrada e não representa, por si só, a medição de impacto ou a leitura das câmeras.

### 4.5 Pontuação e controle de tempo

O tempo inicial configurado é de 30 segundos. Cada evento válido no botão adiciona três segundos e gera pontuação ilustrativa de 100, 70 ou 40 pontos conforme o tempo decorrido desde o surgimento do alvo. O cálculo considera a rapidez do acionamento, não a precisão espacial do movimento da bola.

### 4.6 Encerramento

A função loop() atualiza o tempo restante sem utilizar delay() na rotina principal; quando o tempo termina, a máscara de saída é zerada e os LEDs são desligados. O comando r reinicia a demonstração.

## 5 CONEXÕES DO CÓDIGO DE REFERÊNCIA

| Pino ou terminal | Ligação proposta |
|---|---|
| Arduino D8 | DS/SER do primeiro 74HC595 |
| Arduino D9 | SH_CP/SRCLK dos registradores |
| Arduino D10 | ST_CP/RCLK dos registradores |
| 74HC595 Q7' | DS/SER do registrador seguinte |
| Arduino D2 | Botão de teste para GND (INPUT_PULLUP) |
| OE / MR / alimentação | OE em GND, MR em VCC; VCC e GND conforme o CI |
| Saídas dos registradores | LEDs com resistores limitadores adequados |

As conexões da tabela não foram verificadas ponto a ponto nas fotografias. Recomenda-se confirmar o modelo dos CIs, o consumo dos LEDs, a alimentação comum, o aterramento e a presença de resistores limitadores antes de energizar a montagem. Quando utilizada fonte externa, os referenciais de GND devem ser compatíveis e os limites elétricos de todos os componentes devem ser respeitados.

## 6 EVIDÊNCIAS FOTOGRÁFICAS

As imagens a seguir foram fornecidas pela equipe e mostram etapas da montagem e do acionamento visual do protótipo. Elas demonstram a existência da estrutura eletrônica e a iluminação de LEDs, mas não comprovam isoladamente o funcionamento do botão de teste, a leitura de um sensor, os comandos seriais ou a integração completa com a aplicação.

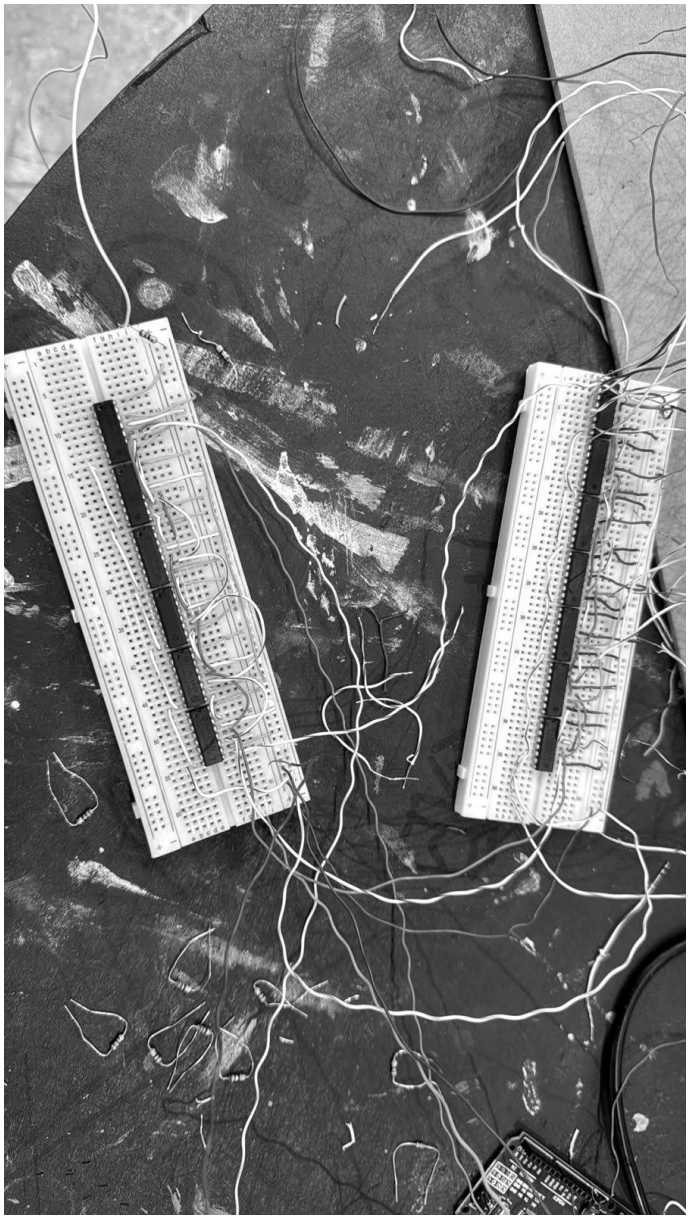


**Figura 1 – Circuitos e conexões montados sobre protoboards.**
**Fonte: acervo da equipe Sweet Spot (2026).**

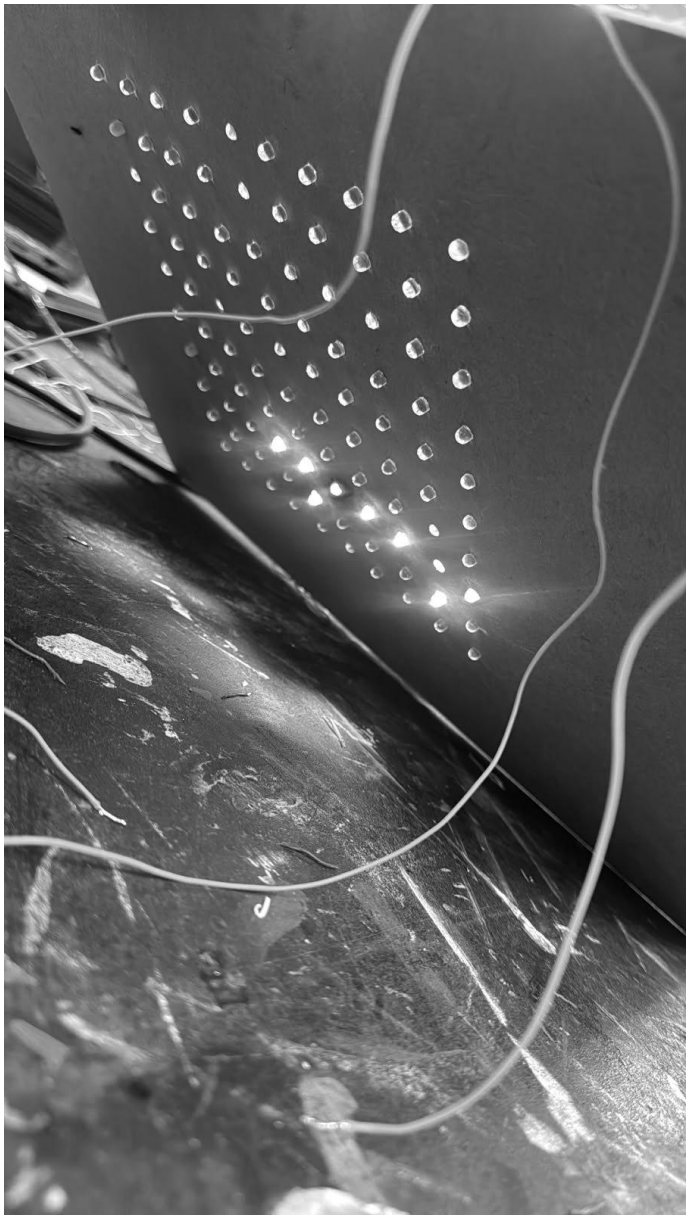


**Figura 2 – LEDs acesos no painel de alvos da maquete.**
**Fonte: acervo da equipe Sweet Spot (2026).**

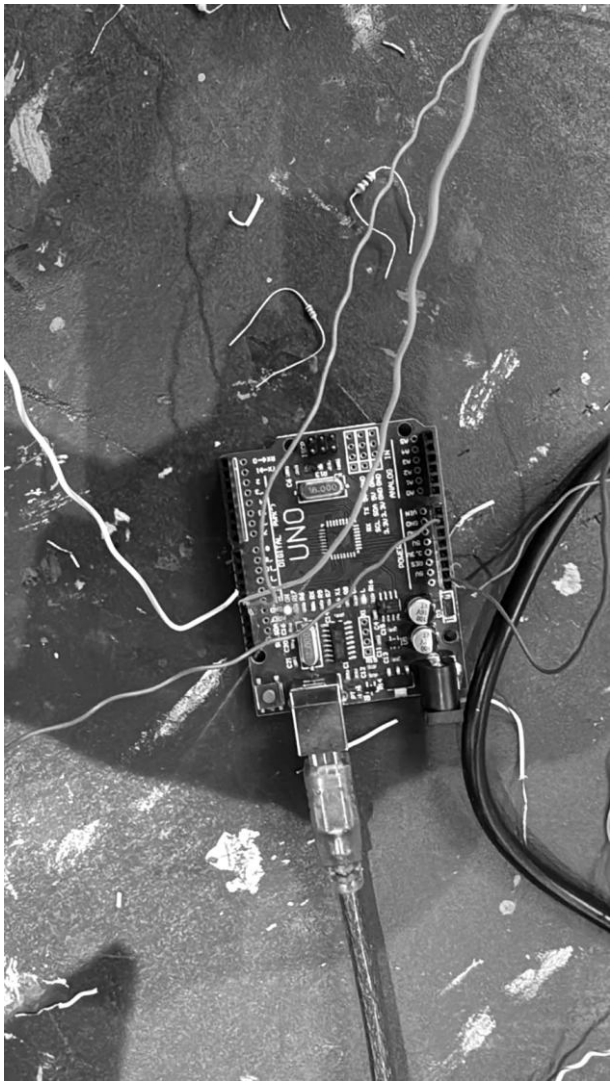



**Figura 3 – Arduino Uno conectado à montagem física.**
**Fonte: acervo da equipe Sweet Spot (2026).**

Link para o video do projeto funcionando:

https://drive.google.com/file/d/12p1CfveEmAp9gtoZEe40plwuV4mUutXS/view?usp=sharing

## 7 PROCEDIMENTO DE VALIDAÇÃO

| Etapa | Procedimento | Evidência esperada |
|---|---|---|
| 1 | Conferir alimentação e ligação dos LEDs. | Alvos iluminam sem aquecimento anormal. |
| 2 | Carregar o código no Arduino IDE. | Compilação e gravação concluídas. |
| 3 | Enviar 1, 2 ou 3 no Monitor Serial. | Tamanho do alvo alterado. |
| 4 | Acionar o botão ligado a D2. | Evento e tempo de reação no Monitor Serial. |
| 5 | Aguardar o fim do cronômetro. | LEDs apagados e aviso de encerramento. |
| 6 | Enviar r. | Treino reiniciado com novo alvo. |

O procedimento é proposto para testes futuros. Sem vídeo de execução, sketch efetivamente carregado e registro do Monitor Serial, não se pode apresentar esses resultados como já verificados no protótipo real.

## 8 RELAÇÃO COM OS REQUISITOS DA ENTREGA

| Requisito | Atendimento documentado |
|---|---|
| Código-fonte comentado | O Apêndice A contém o sketch completo e comentado. |
| Descrição do funcionamento | Seções 3 a 5 descrevem arquitetura, comandos e lógica. |
| Controle de atuador | LEDs acesos registrados na figura 2. |
| Leitura de sensor/entrada | Implementada como botão digital no código demonstrativo; teste físico não comprovado. |
| Evidência prática | Três fotografias da montagem; vídeo de leitura digital recomendado. |

## 9 CONCLUSÃO

A documentação apresenta a proposta de controle luminoso para os alvos da Sweet Spot e registra uma montagem física com Arduino Uno, protoboards e LEDs acesos. O programa demonstrativo contempla os conteúdos de programação abordados na disciplina e fornece uma base de testes para comunicação com registradores de deslocamento e entrada digital.

Para comprovar integralmente o requisito de leitura de sensor, recomenda-se anexar um vídeo curto mostrando a entrada digital acionada e a correspondente mensagem no Monitor Serial, ou documentar o sensor efetivamente utilizado na versão final. A integração com as câmeras e o cálculo de precisão real da bola permanecem em escopo distinto.

## REFERÊNCIAS

ARDUINO. Arduino Language Reference. Disponível em: https://docs.arduino.cc/language-reference/. Acesso em: 9 nov. 2026.

SWEET SPOT. Entrega 01 – Teoria da Computação e Linguagens Formais. Material da equipe, 2026.

SWEET SPOT. Registros fotográficos do protótipo. Acervo da equipe, 2026.

## APÊNDICE A – CÓDIGO-FONTE COMENTADO DO ARDUINO

O código a seguir reproduz integralmente o arquivo demonstrativo SweetSpot_Entrega02_Arduino.ino. As ligações indicadas nos comentários são propostas e devem ser confrontadas com o circuito efetivamente utilizado. O programa trabalha com 16 saídas e não representa automaticamente toda a matriz física das fotografias.

```
/*
  Sweet Spot - Entrega 02: Programacao e Controle de Dispositivos
  Demonstracao didatica: 16 LEDs via dois 74HC595 e um botao como sensor digital.
  IMPORTANTE: o botao representa um evento de acerto para testes; ele NAO mede
  impacto, trajetoria, posicao real da bola nem precisao espacial.

  Ligacoes propostas (confirmar com a montagem fisica):
  Arduino D8  -> DS / SER do primeiro 74HC595
  Arduino D9  -> SH_CP / SRCLK de ambos os 74HC595
  Arduino D10 -> ST_CP / RCLK de ambos os 74HC595
  Q7' do primeiro -> DS / SER do segundo 74HC595 (cascata)
  Arduino D2 -> botao -> GND (INPUT_PULLUP)
  74HC595: OE -> GND; MR / SRCLR -> VCC; VCC -> 5 V; GND -> GND
  Cada LED deve ter resistor limitador individual, respeitando os limites
  de corrente do CI e da alimentacao. Usar desacoplamento de 100 nF por CI.
*/

const byte PIN_DADOS = 8;
const byte PIN_CLOCK = 9;
const byte PIN_LATCH = 10;
const byte PIN_SENSOR = 2;

const byte TOTAL_LEDS = 16;
byte larguraAlvo = 4;                 // medio: quatro LEDs ligados
byte inicioAlvo = 0;
unsigned long pontuacao = 0;
unsigned long acertos = 0;
unsigned long instanteAlvo = 0;
unsigned long tempoRestanteMs = 30000UL;

const unsigned long BONUS_MS = 3000UL;
const unsigned long DEBOUNCE_MS = 35UL;
unsigned long instanteMudanca = 0;
bool leituraAnterior = HIGH;
bool estadoEstavel = HIGH;
bool treinoAtivo = true;
unsigned long ultimoLoop = 0;

// Envia uma mascara de 16 bits aos dois registradores de deslocamento.
void escreverLeds(uint16_t mascara) {
  digitalWrite(PIN_LATCH, LOW);
  shiftOut(PIN_DADOS, PIN_CLOCK, MSBFIRST, highByte(mascara));
  shiftOut(PIN_DADOS, PIN_CLOCK, MSBFIRST, lowByte(mascara));
  digitalWrite(PIN_LATCH, HIGH);
}

void gerarAlvo() {
  inicioAlvo = random(0, TOTAL_LEDS - larguraAlvo + 1);
  uint16_t mascara = 0;
  for (byte i = 0; i < larguraAlvo; i++) {
    mascara |= (uint16_t)1 << (inicioAlvo + i);
  }
  escreverLeds(mascara);
  instanteAlvo = millis();
  Serial.print(F("Novo alvo: LEDs "));
  Serial.print(inicioAlvo);
  Serial.print(F(" a "));
  Serial.println(inicioAlvo + larguraAlvo - 1);
}
```

```
void reiniciarTreino() {
  acertos = 0;
  pontuacao = 0;
  tempoRestanteMs = 30000UL;
  treinoAtivo = true;
  ultimoLoop = millis();
  gerarAlvo();
  Serial.println(F("Treino reiniciado (30 s)."));
}

void registrarEventoSensor() {
  if (!treinoAtivo) return;
  unsigned long reacaoMs = millis() - instanteAlvo;
  // Pontuacao demonstrativa por rapidez; nao e precisao real da bola.
  unsigned int pontos = reacaoMs < 1000UL ? 100 :
                        reacaoMs < 2000UL ? 70 : 40;
  acertos++;
  pontuacao += pontos;
  tempoRestanteMs += BONUS_MS;
  Serial.print(F("Evento do sensor | Reacao: "));
  Serial.print(reacaoMs);
  Serial.print(F(" ms | Pontos: "));
  Serial.print(pontos);
  Serial.print(F(" | Total: "));
  Serial.print(pontuacao);
  Serial.print(F(" | Acertos simulados: "));
  Serial.println(acertos);
  gerarAlvo();
}

void setup() {
  pinMode(PIN_DADOS, OUTPUT);
  pinMode(PIN_CLOCK, OUTPUT);
  pinMode(PIN_LATCH, OUTPUT);
  pinMode(PIN_SENSOR, INPUT_PULLUP);
  Serial.begin(9600);
  randomSeed(analogRead(A0)); // A0 desconectado e usado somente como semente
  Serial.println(F("Sweet Spot | 1 facil, 2 medio, 3 dificil, r reiniciar"));
  reiniciarTreino();
}

void loop() {
  unsigned long agora = millis();
  unsigned long delta = agora - ultimoLoop;
  ultimoLoop = agora;

  if (treinoAtivo) {
    if (delta >= tempoRestanteMs) {
      tempoRestanteMs = 0;
      treinoAtivo = false;
      escreverLeds(0);
      Serial.println(F("Fim do treino. Pressione r para reiniciar."));
    } else {
      tempoRestanteMs -= delta;
    }
  }

  // Leitura do sensor digital com debounce e deteccao de borda.
  bool leitura = digitalRead(PIN_SENSOR);
  if (leitura != leituraAnterior) instanteMudanca = agora;
  if (agora - instanteMudanca >= DEBOUNCE_MS && leitura != estadoEstavel) {
    estadoEstavel = leitura;
    if (estadoEstavel == LOW) registrarEventoSensor();
  }
  leituraAnterior = leitura;

  // Ajuste de dificuldade pelo Monitor Serial.
  if (Serial.available() > 0) {
    char comando = Serial.read();
    if (comando == '1') larguraAlvo = 6; // facil: alvo maior
    if (comando == '2') larguraAlvo = 4; // medio
    if (comando == '3') larguraAlvo = 2; // dificil: alvo menor
    if (comando == 'r' || comando == 'R') {
      reiniciarTreino();
```

```
    } else if (comando >= '1' && comando <= '3') {
      Serial.print(F("Dificuldade alterada. Largura: "));
      Serial.println(larguraAlvo);
      if (treinoAtivo) gerarAlvo();
    }
  }
}
```